Universidade Federal do Amazonas – UFAM
Programa de Pós-Graduação (Mestrado)

Matheus Serrão Uchoa

Verificação formal de um controlador neural em malha fechada (*cart-pole*): resultados com BMC (ESBMC) e com IC3/PDR

Manaus
2026

Matheus Serrão Uchoa

Verificação formal de um controlador neural em malha fechada (*cart-pole*): resultados com BMC (ESBMC) e com IC3/PDR

Relatório técnico de resultados do projeto de verificação formal de controladores neurais quantizados, comparando *Bounded Model Checking* (ESBMC) e *model checking* indutivo (IC3/PDR) sobre o ator DDPG do problema do pêndulo invertido sobre carro.

Manaus

2026

RESUMO

Este relatório consolida os resultados da verificação formal de um controlador neural em malha fechada: o ator de um agente DDPG (arquitetura 4→24→24→1, 745 parâmetros, quantizado em ponto fixo Q8.8) que equilibra o pêndulo invertido sobre carro (*cart-pole*). Duas famílias de método são comparadas. A primeira usa *Bounded Model Checking* (BMC) com o ESBMC 6.8.0 sobre *harnesses* em C; ela decide propriedades de um e de dois passos, encontra contraexemplos concretos e reproduzíveis, mas o custo cresce de 0,3 s (K=1) para 909 s (K=4) e a prova de segurança para horizonte ilimitado fica fora de alcance. A segunda traduz a malha fechada para um sistema de transição (Verilog → AIGER, 904 242 portas AND, 65 *latches*) e aplica IC3/PDR (ABC); ela produz uma prova ilimitada num modelo sintético com batente mecânico, mas não decide o ator real: o PDR parou no *frame* 2 após 1800 s, o BMC do ABC alcançou 5 *frames* e um motor independente em nível de palavra (Z3/Spacer) também não decidiu. A validação do pipeline mostra que a tradução é equivalente ao C por análise de intervalos e por prova SAT composicional em 49 de 50 blocos, que o pipeline reprova três mutantes comprovadamente inseguros e que as lacunas restantes são declaradas. O relatório **não** afirma que o controlador real seja seguro para sempre: afirma o que foi provado, o que foi refutado e o que permanece indeciso.

Palavras-chave: verificação formal; *model checking*; ESBMC; IC3; PDR; redes neurais quantizadas; *cart-pole*; DDPG.

SUMÁRIO

1 Introdução 7
1.1 Contexto 7
1.2 Problema 7
1.3 Motivação (Justificativas) 7
1.4 Objetivos 8
1.4.1 Objetivo geral 8
1.4.2 Objetivos específicos 8
2 Trabalhos Correlatos 9
2.1 Verificação de redes neurais por SMT, MILP e alcançabilidade 9
2.2 Redes quantizadas 9
2.3 *Model checking* de controladores neurais 9
2.4 IC3/PDR e motores indutivos 9
2.5 Lacuna e posicionamento 10
3 Referencial Teórico 11
3.1 O sistema: pêndulo invertido sobre carro 11
3.2 Ponto fixo Q8.8 e divisão por termo 11
3.3 BMC e ESBMC 12
3.4 IC3/PDR 12
3.5 AIGER, bit-*blasting* e nível de palavra 12
4 Método Proposto 14
4.1 Visão geral 14
4.2 Convenção de vereditos 14
4.2.0.0.1 Ambiente. 15
4.3 Rota BMC: o *cart-pole* verificado com ESBMC 15
4.3.1 Método 15
4.3.1.0.1 Extração e geração de *harnesses*. 15
4.3.1.0.2 Domínio de entrada. 15
4.3.1.0.3 Propriedades. 15
4.3.2 Resultados do ator DDPG 15
4.3.2.0.1 Leitura do contraexemplo de A. 16
4.3.3 Propriedades vacuas e sua correção 16
4.3.4 Envelope de segurança em dois passos 17
4.3.5 A parede do BMC 17
4.3.6 Propriedades estruturais da rede 17
4.3.7 O controlador DQN, para comparação 18

4.3.8 Síntese da Parte I . . . 18
4.4 Rota indutiva: IC3/PDR . . . 18
4.4.1 Do C ao sistema de transição . . . 18
4.4.2 Resultados sobre o ator real . . . 19
4.4.2.0.1 O sucesso sintético. . . . 19
4.4.2.0.2 O ator real. . . . 20
4.4.3 Comparação controlada sobre a mesma instância . . . 20
4.4.3.0.1 O que a Tabela 5 diz. . . . 21
4.5 Validação do pipeline . . . 21
4.5.1 Fidelidade da tradução . . . 21
4.5.1.1 O lema da divisão . . . 21
4.5.1.2 Análise de intervalos . . . 21
4.5.1.3 Equivalência por SAT . . . 21
4.5.1.3.1 Ausência de *wrap* de 16 bits. . . . 22
4.5.2 Teste de mutação: o pipeline sabe reprovar? . . . 22
4.5.3 Motor independente em nível de palavra . . . 22
4.6 Resultados preliminares: discussão e ameaças à validade . . . 23
4.6.1 Balanço de evidência . . . 23
4.6.2 Isto é revolucionário? . . . 24
4.6.3 Ameaças à validade . . . 24
5 Entregáveis . . . 25
6 Metodologia e Cronograma . . . 26
6.1 Metodologia . . . 26
6.2 Atividades . . . 26
6.3 Cronograma . . . 26
7 Conclusão . . . 28

Apêndices 29
APÊNDICE A Logs brutos selecionados . . . 30
A.1 PDR no ator real (excerto de `real/cl_ddpg16.abc.out`) . . . 30
A.2 Resumo do *wrapper* (`real/run_pdr.stdout.txt`) . . . 30
A.3 Contraexemplo do lema da divisão (`lema_td.cex.txt`) . . . 30
A.4 Mutante detectado (`mut/mutantes.saida.txt`) . . . 30
APÊNDICE B Reprodução . . . 31
Referências . . . 32

LISTA DE ILUSTRAÇÕES

Figura 1 – Malha fechada verificada: o ator (rede quantizada) lê o estado, o tanh por partes satura a força em ±10 N e a dinâmica linearizada produz o próximo estado. 11
Figura 2 – Tempo do BMC (ESBMC) versus horizonte K, escala logarítmica. . . . 18
Figura 3 – As duas rotas de verificação a partir do mesmo ator quantizado. 19
Figura 4 – Tamanho do problema (portas AND do AIG) versus largura H das camadas ocultas. 20

LISTA DE TABELAS

Tabela 1 – Verificação do ator DDPG com ESBMC 6.8.0: execução registrada. . . 16
Tabela 2 – Segurança em dois passos em função do limite de $|\dot{\theta}|$ inicial. 17
Tabela 3 – Custo do BMC em função do horizonte K. 17
Tabela 4 – BMC e IC3 sobre a malha fechada. 19
Tabela 5 – Comparação controlada, mesma instância, 300 s por motor. 20
Tabela 6 – Resumo da fidelidade da tradução. 22
Tabela 7 – Mutantes do ator real. 23
Tabela 8 – O que está estabelecido, o que está refutado e o que está aberto. 23
Tabela 9 – Atividades planejadas (em ordem de custo–benefício). 27
Tabela 10 – Cronograma proposto das atividades restantes (“X” marca o mês de execução). 27

1 INTRODUÇÃO

1.1 Contexto

Redes neurais profundas passaram a atuar como controladores em sistemas físicos: um agente de aprendizado por reforço (por exemplo, DDPG; LILLICRAP et al., 2016) lê o estado da planta e produz diretamente a ação de controle. Em sistemas embarcados, a rede é executada em aritmética de ponto fixo inteira, o que reduz custo e latência, mas introduz truncamentos e saturações que não existem no modelo em ponto flutuante usado no treinamento.

O problema clássico usado como *benchmark* neste trabalho é o pêndulo invertido sobre carro (*cart-pole*; BARTO; SUTTON; ANDERSON, 1983). O estado é $\mathbf{s} = [x, \dot{x}, \theta, \dot{\theta}]^\top$ (posição e velocidade do carro, ângulo e velocidade angular do pêndulo) e a ação é uma força horizontal limitada a ± 10 N. A operação é considerada segura enquanto $|\theta| \leq 12^\circ$ (e $|x| \leq 2{,}4$ m).

1.2 Problema

A pergunta de segurança é universal: *para todo estado inicial admissível, e para todo instante t, o controlador neural quantizado em malha fechada mantém $|\theta| \leq 12^\circ$?* Simulação e teste só cobrem trajetórias amostradas e não respondem a essa pergunta. Verificação formal pode responder, mas enfrenta dois obstáculos neste caso:

1. o *Bounded Model Checking* (BMC) prova segurança apenas até um horizonte K, e seu custo cresce rapidamente com K (medido aqui: 0,3 s em K=1 e 909 s em K=4);
2. o *model checking* indutivo (IC3/PDR) pode provar segurança sem horizonte, mas não é sabido se converge para uma rede com 745 parâmetros e aritmética inteira truncada.

O problema de pesquisa é, portanto, determinar *até onde* BMC e IC3/PDR decidem a segurança do controlador real e como tornar essa conclusão confiável (tradução fiel, propriedades não vazias, capacidade de reprovar).

1.3 Motivação (Justificativas)

Uma prova de segurança só vale se (i) a propriedade depende de fato do controlador, (ii) o modelo verificado é equivalente ao artefato executado e (iii) o verificador é capaz de

reprovar modelos inseguros. Os resultados preliminares mostram que esses três pontos não são triviais: duas das três provas originais do BMC eram vazias (não dependiam dos pesos da rede); a tradução para o sistema de transição tem uma divergência (divisão de -2^{31}) e lacunas de equivalência declaradas; e a rota IC3 só produziu uma prova ilimitada num modelo com batente mecânico, não no controlador real. Documentar e fechar essas lacunas é necessário antes de qualquer afirmação de segurança, e o registro honesto de resultados negativos é, ele próprio, útil a quem pretenda verificar redes quantizadas em malha fechada.

1.4 Objetivos

1.4.1 Objetivo geral

Avaliar, de forma reproduzível e auditável, a capacidade do BMC (ESBMC) e do IC3/PDR de verificar a segurança de um controlador neural quantizado (ator DDPG, Q8.8) em malha fechada no *cart-pole*, e estabelecer o que está provado, refutado e indeciso.

1.4.2 Objetivos específicos

1. Formalizar a malha fechada (rede quantizada, tanh por partes e dinâmica linearizada) e as propriedades de segurança, isolando propriedades vacuas.
2. Verificar propriedades locais e de curto horizonte com ESBMC e caracterizar o custo do BMC em função do horizonte.
3. Traduzir a malha fechada para um sistema de transição (Verilog $\rightarrow$ AIGER) e aplicar IC3/PDR, comparando-o com BMC e com um motor em nível de palavra.
4. Validar o pipeline: fidelidade da tradução (intervalos e equivalência por SAT) e capacidade de reprovar (mutantes de insegurança conhecida).
5. Fechar as lacunas restantes: equivalência de `TANH_DIN`, ausência de *wrap* de 16 bits e validação da planta linearizada.
6. Posicionar o trabalho frente à literatura de verificação de redes neurais.

2 TRABALHOS CORRELATOS

Aviso metodológico. Esta seção é um levantamento preliminar. Parte das referências foi identificada apenas por título e resumo, **sem leitura integral**; os dados bibliográficos marcados com "(a conferir)" devem ser verificados antes da versão final. Uma revisão sistemática é uma atividade do cronograma (Capítulo 4).

2.1 Verificação de redes neurais por SMT, MILP e alcançabilidade

O *Reluplex* (KATZ et al., 2017) estendeu o simplex para tratar ReLU e verificar propriedades de redes; seu sucessor, o *Marabou* (KATZ et al., 2019), é um *framework* de SMT para redes. O *NNV* (TRAN et al., 2020) usa alcançabilidade com conjuntos do tipo estrela. O α, β-CROWN (WANG et al., 2021) combina propagação de limites com *branch-and-bound* e lidera competições de verificação. Esses verificadores tratam a rede *isolada* (propriedade entrada–saída), em geral em ponto flutuante; a malha fechada com dinâmica discreta e aritmética inteira truncada por termo não é o seu foco.

2.2 Redes quantizadas

A verificação de redes quantizadas por SMT/BMC é tratada, por exemplo, em arXiv:2106.05997 (autoria a conferir), e formulações em programação linear inteira para redes quantizadas (QVIP, a conferir). Esses trabalhos tratam a semântica de bit-vetores da rede, que é a mesma preocupação deste trabalho, mas, pelo que foi levantado, sem a planta em malha fechada.

2.3 *Model checking* de controladores neurais

Trabalhos recentes aplicam *model checking* a sistemas com componentes neurais (por exemplo, Giacobbe et al., NeurIPS 2024, a conferir). Em verificação de software, o ESBMC (GADELHA et al., 2018) é usado para programas C, inclusive código gerado de redes; e a verificação de C via BTOR2 com IC3/PDR é uma rota estabelecida.

2.4 IC3/PDR e motores indutivos

O IC3 (BRADLEY, 2011) e a implementação PDR do ABC (EÉN; MISHCHENKO; BRAYTON, 2011) são a base dos *model checkers* de *hardware*; o Spacer (KOMURAVELLI; GURFINKEL; CHAKI, 2014) generaliza a ideia para cláusulas de Horn no Z3. Motores em

nível de palavra para BTOR2 (btormc, Pono, AVR, rIC3) preservam a estrutura aritmética e são a alternativa natural ao *bit-blasting*.

2.5 Lacuna e posicionamento

Numa busca curta não encontrei o uso de IC3/PDR sobre a malha fechada planta mais rede neural quantizada em ponto fixo, com validação por mutantes. Isso **não** equivale a ausência na literatura; a revisão sistemática planejada deve confirmá-lo ou refutá-lo. A contribuição defensável até aqui é metodológica: detecção de propriedades vacuas, cadeia de validação e registro de resultados negativos.

3 REFERENCIAL TEÓRICO

3.1 O sistema: pêndulo invertido sobre carro

O estado é $\mathbf{s} = [x, \dot{x}, \theta, \dot{\theta}]^\top$: posição do carro (m), velocidade (m/s), ângulo do pêndulo em relação à vertical (rad) e velocidade angular (rad/s). A ação é uma força horizontal $F \in [-10, +10]$ N. Os parâmetros usuais são $g = 9{,}8$ m/s^2, massa do carro 1,0 kg, massa do pêndulo 0,1 kg, semicomprimento 0,5 m e passo $\Delta t = 0{,}02$ s (BARTO; SUTTON; ANDERSON, 1983). A falha ocorre quando $|x| > 2{,}4$ m ou $|\theta| > 12°$.

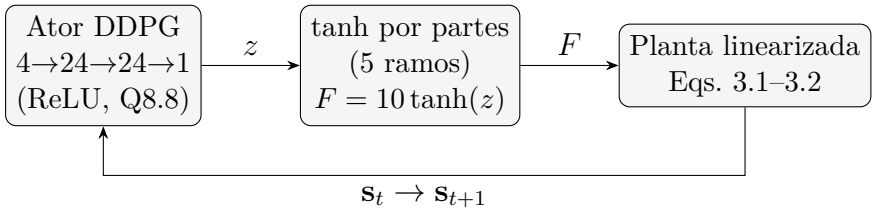
Para verificação usa-se a dinâmica **linearizada** ($\sin\theta \approx \theta$, $\cos\theta \approx 1$), válida no domínio seguro $|\theta| \leq 12° \approx 0{,}209$ rad. Em Q8.8 (Seção 3.2) a atualização de um passo, tal como implementada no gerador, é

$$\ddot{\theta}_Q = \left\lfloor \frac{4040\,\theta_Q - 375\,F_Q}{256} \right\rfloor, \qquad \theta'_Q = \theta_Q + \left\lfloor \frac{5\,\dot{\theta}_Q}{256} \right\rfloor, \tag{3.1}$$

$$\dot{\theta}'_Q = \dot{\theta}_Q + \left\lfloor \frac{5\,\ddot{\theta}_Q}{256} \right\rfloor, \qquad \dot{x}'_Q = \dot{x}_Q + \left\lfloor \frac{5\,F_Q}{256} \right\rfloor, \quad x'_Q = x_Q + \left\lfloor \frac{5\,\dot{x}_Q}{256} \right\rfloor, \tag{3.2}$$

onde $\lfloor \cdot \rfloor$ denota a divisão inteira que trunca em direção a zero (a de C). O fator $5/256 \approx 0{,}0195$ aproxima o passo 0,02 s. O coeficiente $4040/256 \approx 15{,}8$ torna a origem instável em malha aberta: sem controle, θ cresce exponencialmente.

Figura 1 – Malha fechada verificada: o ator (rede quantizada) lê o estado, o tanh por partes satura a força em ±10 N e a dinâmica linearizada produz o próximo estado.



Fonte: elaborada pelo autor, 2026.

3.2 Ponto fixo Q8.8 e divisão por termo

Cada valor real v vira o inteiro $q(v) = \mathrm{round}(256\,v)$. O produto de dois valores Q8.8 tem escala 256^2 e precisa ser dividido por 256. O ator implementa essa divisão **por termo**:

$$\mathrm{pre}_i = \left\lfloor \tfrac{a\,x}{256} \right\rfloor + \left\lfloor \tfrac{b\,\dot{x}}{256} \right\rfloor + \left\lfloor \tfrac{c\,\theta}{256} \right\rfloor + \left\lfloor \tfrac{d\,\dot{\theta}}{256} \right\rfloor + b_i,$$

e não soma-depois-divide, porque os resultados diferem; e a divisão trunca a zero (como o `/` de C), não para $-\infty$ (como o » aritmético), o que muda o resultado para valores negativos. A aproximação linear de tanh em cinco ramos satura em 255 (isto é, ≈ 1) e limita $|F_Q| \leq 2550 < 2560$.

A rede tem $4{\cdot}24 + 24 + 24{\cdot}24 + 24 + 24 + 1 = 745$ parâmetros inteiros. O erro de quantização medido em 10 000 estados (arquivo `quantization_report.json`) é de até 7,76 N no pior caso, com média 0,078 N, percentil 95 de 0,23 N e percentil 99 de 1,09 N. A verificação é sobre o controlador *quantizado*, que é o artefato executado.

3.3 BMC e ESBMC

No *Bounded Model Checking* (BIERE et al., 1999), dado um programa, uma propriedade e um limite K, constrói-se uma fórmula Φ_K satisfatível se e somente se existe uma execução de até K passos que viola a propriedade. O ESBMC (GADELHA et al., 2018) traduz programas C para fórmulas SMT. Satisfatível implica contraexemplo concreto; insatisfatível implica seguro *até* K. A fórmula contém K **cópias** da relação de transição, o que explica o crescimento de custo com K (Seção 4.3.5). As primitivas usadas nos *harnesses* são `nondet_int()` (variável simbólica), `__ESBMC_assume` (restringe o espaço) e `__ESBMC_assert` (propriedade).

3.4 IC3/PDR

O IC3 (BRADLEY, 2011), na implementação PDR do ABC (EÉN; MISHCHENKO; BRAYTON, 2011), mantém uma sequência de *frames* $F_0, F_1, \ldots$ que sobreaproximam os estados alcançáveis em até i passos e os refina com cláusulas aprendidas, cada uma obtida por uma consulta SAT sobre **uma única cópia** da transição. Se dois *frames* consecutivos coincidem, tem-se um **invariante indutivo**: o conjunto contém os estados iniciais, é fechado pela transição e não toca a região insegura. Isso prova segurança para todo t, sem horizonte. O IC3 pode não terminar: sem boa generalização das cláusulas, o número de *frames* e de cláusulas cresce sem convergir.

3.5 AIGER, bit-*blasting* e nível de palavra

O fluxo de síntese usa Yosys (WOLF; GLASER; KEPLER, 2013) e ABC (BRAYTON; MISHCHENKO, 2010): o Verilog é reduzido a um *And-Inverter Graph* (AIG). Nessa forma cada palavra de 16 bits vira 16 bits independentes e a estrutura aritmética se perde; é uma hipótese deste trabalho, **não demonstrada**, que isso prejudique a generalização do IC3. Um motor em *nível de palavra* (BTOR2: btormc, Pono, AVR, rIC3) preservaria a

estrutura; nenhum deles foi executado aqui. Como substituto parcial usou-se o Z3/Spacer (resolvedor de cláusulas de Horn; KOMURAVELLI; GURFINKEL; CHAKI, 2014; DE MOURA; BJØRNER, 2008) sobre aritmética inteira exata.

4 MÉTODO PROPOSTO

4.1 Visão geral

O método proposto é um *pipeline* em quatro etapas, executado com as mesmas instâncias nas duas rotas de verificação (Figura 3):

1. **Especificação:** controlador quantizado em Q8.8, tanh por partes, dinâmica linearizada, região inicial e propriedades (A–D);
2. **Rota BMC:** *harness* em C e ESBMC, para contraexemplos e prova até o horizonte K;
3. **Rota indutiva:** sistema de transição em Verilog, AIGER (Yosys) e IC3/PDR (ABC), com motores alternativos (BMC do ABC, Z3/Spacer, e BTOR2 em nível de palavra, planejado);
4. **Validação:** intervalos, equivalência por SAT, mutantes de insegurança conhecida e isolamento de propriedades vacuas.

As seções seguintes descrevem cada etapa e apresentam os **resultados preliminares** já obtidos com ela. A etapa de BTOR2 e as demais lacunas são tratadas como trabalho proposto na Seção 4.6 e no Capítulo 6.

4.2 Convenção de vereditos

Todo resultado cai em exatamente uma de quatro categorias, e nenhuma é substituída por outra:

- **Provado**: o motor concluiu que a propriedade vale (no escopo e na codificação declarados);
- **Contraexemplo**: o motor exibiu uma execução concreta que viola a propriedade;
- **Indeciso**: o motor excedeu o limite de tempo ou de memória. **Indeciso não é seguro e não é inseguro**;
- **Modelo não validado**: o resultado vale para o modelo traduzido; a equivalência com o artefato real tem lacunas declaradas.

4.2.0.0.1 Ambiente.

As medições da validação e as reexecuções da rota indutiva usaram Yosys 0.33 (ABC embutido), Z3 4.8.12, contêiner Ubuntu 24.04, Intel i7-14700 (28 *threads*) e 32 GB de RAM. As medições do ESBMC 6.8.0 são as registradas no repositório do projeto; tempos absolutos entre os dois conjuntos não são comparáveis.

4.3 Rota BMC: o *cart-pole* verificado com ESBMC

4.3.1 Método

4.3.1.0.1 Extração e geração de *harnesses*.

Um *pipeline* Python extrai os pesos do ator, quantiza em Q8.8, propaga intervalos de pré-ativação e emite um programa C sem laços: cada neurônio é uma linha, com ReLU como `pre > 0 ? pre : 0`. A ausência de laços permite `--no-unwinding-assertions` sem perda de solidez.

4.3.1.0.2 Domínio de entrada.

Em Q8.8: $|x| \leq 614$ (2,4 m), $|\dot{x}| \leq 1280$ (5 m/s), $|\theta| \leq 53$ (12°), $|\dot{\theta}| \leq 1280$ (5 rad/s), declarados por `__ESBMC_assume`.

4.3.1.0.3 Propriedades.

Quatro propriedades foram escritas para o ator DDPG:

- **A** (direção): na zona de perigo (θ grande e $\dot{\theta}$ no mesmo sentido) o controlador empurra para o lado que corrige;
- **B** (segurança): a partir do domínio seguro, $|\theta| \leq 12°$ após N passos;
- **C** (limite de força): $|F_Q| \leq 2560$;
- **D** (não trivialidade): na zona de perigo o controlador aplica força de módulo mínimo (0,5 N).

4.3.2 Resultados do ator DDPG

A Tabela 1 reproduz o registro de execuções do ator DDPG com ESBMC 6.8.0.

Tabela 1 – Verificação do ator DDPG com ESBMC 6.8.0: execução registrada.

Propriedade	**Veredito**	**Tempo**	**Contraexemplo / observação**
A (lado direito)	Contraexemplo	1888 s	x= − 0,0117, $\dot{x}$= − 4,92, θ=0,109, $\dot{\theta}$=0
A (lado esquerdo)	Contraexemplo	2722 s	$z = 27$ (força positiva com $\theta < -5{,}7°$)
B (1 passo)	Contraexemplo	90 s	x= − 0,754, $\dot{x}$= − 3,92, θ= − 0,184, $\dot{\theta}$= − 1,52 (propriedade vacua; §4.3.3)
C (limite de força)	Provado	24 s	vale por construção do tanh (§4.3.3)

Fonte: `cartpole/ddpg_closed_loop_results.json`. Execução anterior às correções da §4.3.3; a propriedade D não tem resultado armazenado nesse arquivo.

4.3.2.0.1 Leitura do contraexemplo de A.

O ESBMC encontrou estados em que a rede aplica a força contrária à esperada, o que é uma violação da propriedade *como especificada*. Uma ressalva: $z > 0$ não implica $F > 0$, pois a divisão inteira do primeiro ramo do tanh dá $\lfloor 1 \cdot 252/256 \rfloor = 0$; logo $z = 1$ resulta em força nula. A propriedade A é, portanto, ao mesmo tempo restritiva e frouxa; a propriedade D foi criada para cobrir a lacuna.

4.3.3 Propriedades vacuas e sua correção

Duas propriedades originais não dependiam do controlador, o que foi descoberto ao executar o mesmo *assert* sem a rede:

1. **B de um passo.** O valor asseriado era $\theta_{novo} = \theta + 5\dot{\theta}/256$, que *não contém* F (Equações 3.1–3.2: a força só chega a θ via $\dot{\theta}$, no passo seguinte). Executando sem rede e com F_Q arbitrário, o veredito e o contraexemplo ($\theta = 53$, $\dot{\theta} = 64 \rightarrow \theta_{novo} = 54$) se reproduzem idênticos: os 745 pesos eram código morto. **Correção:** B passou a dois passos, asserindo $\theta_2 = \theta_{novo} + 5\dot{\theta}_{novo}/256$. Dependência confirmada: com F_Q livre o contraexemplo é $\theta_2 = 79$; com $F_Q = 0$ é $\theta_2 = 55$.

2. **C.** $|F_Q| \leq 2560$ vale por construção, pois o tanh por partes satura em 255 e $255 \cdot 10 = 2550$. Substituindo a rede inteira por z livre, a prova passa em 0,72 s. **Correção:** C foi reclassificada como *sanidade da quantização*, e D foi criada para depender dos pesos.

Este é um resultado metodológico: duas das três provas originais eram vazias, e só foi possível saber isso ao isolar a propriedade da rede.

4.3.4 Envelope de segurança em dois passos

Tabela 2 – Segurança em dois passos em função do limite de $|\dot{\theta}|$ inicial.

$\lvert\dot{\theta}\rvert \leq$ (Q8.8)	rad/s	Veredito	Tempo
1280	5,00	Contraexemplo	25,4 s
640	2,50	Contraexemplo	28,8 s
320	1,25	Indeciso	>300 s
160	0,62	Indeciso	>300 s
80	0,31	Indeciso	>300 s

Fonte: capítulo de resultados do artigo do projeto (`artigo/caps/4resultados.tex`, Tabela `envelope`).

Conclusão defensável: a segurança de dois passos é **violada** para $|\dot{\theta}| \geq 2{,}5$ rad/s e **indecisa** abaixo disso. Não se conclui que o controlador seja seguro nas caixas estreitas. O comportamento é contraintuitivo: *reduzir a região inicial torna o problema mais difícil*. Com caixa larga existe contraexemplo, e achá-lo é a direção barata (SAT); com caixa estreita é preciso *provar* que não existe, e a prova (UNSAT) é cara.

4.3.5 A parede do BMC

A Tabela 3 e a Figura 2 mostram a medição com uma rede de 24 neurônios.

Tabela 3 – Custo do BMC em função do horizonte K.

K	Tempo	Memória
1	0,3 s	46 MB
2	77 s	137 MB
4	909 s	383 MB

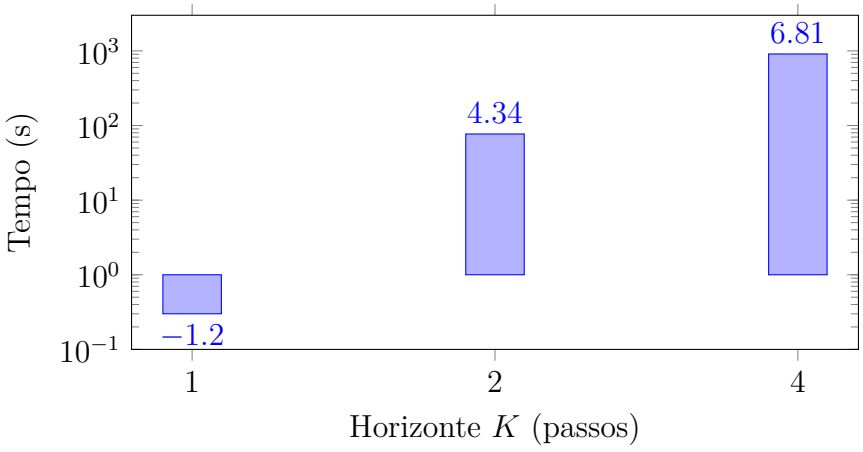
Fonte: cabeçalho de `ic3/gen_transition_system.py`.

O *harness* de 50 passos nunca foi concluído por esse motivo: a parede está em K=4, não em 50. A causa é estrutural (uma única fórmula com K cópias da transição), não de esforço.

4.3.6 Propriedades estruturais da rede

Perguntou-se ao ESBMC se algum neurônio é *morto* (ReLU nunca ativa) ou *saturado*. No ator DDPG, o registro (`ddpg_dead_neuron_results.json`, `ddpg_saturation_results.json`) indica: **nenhum dos 48 neurônios é morto** (24 por camada, sem *timeouts*), nenhum neurônio da camada 1 é saturado e a saída é responsiva. Ressalva: na camada 2 a entrada h_1 é tratada como caixa relaxada (não o valor calculado a partir do estado); assim, "vivo" na camada 2 deve ser lido com cautela e o resultado firme vale para a camada 1.

Figura 2 – Tempo do BMC (ESBMC) versus horizonte K, escala logarítmica.



Fonte: Tabela 3.

4.3.7 O controlador DQN, para comparação

O projeto também verificou um controlador DQN (`closed_loop_results.json`). A propriedade A obteve contraexemplo no lado esquerdo (x= − 1,246, $\dot{x}$=3,52, θ= − 0,191, $\dot{\theta}$= − 0,42, ação 1) e *timeout* no lado direito. A segurança de um passo (B) obteve contraexemplo em $\dot{\theta} = -4{,}59$ rad/s, um valor extremo em que θ cruza o limite em um passo independentemente da ação, o que *reforça* o achado de vacuidade da Seção 4.3.3.

4.3.8 Síntese da Parte I

O ESBMC **decide** propriedades locais (neurônios, limites, um e dois passos com caixas largas) e produz contraexemplos concretos, reproduzíveis no simulador. Ele **não decide** (i) a segurança de dois passos em caixas estreitas e (ii) horizontes $K \gtrsim 4$. A prova de segurança para $t \to \infty$ está fora do alcance do método nesta formulação, o que motiva a rota indutiva do capítulo seguinte.

4.4 Rota indutiva: IC3/PDR

4.4.1 Do C ao sistema de transição

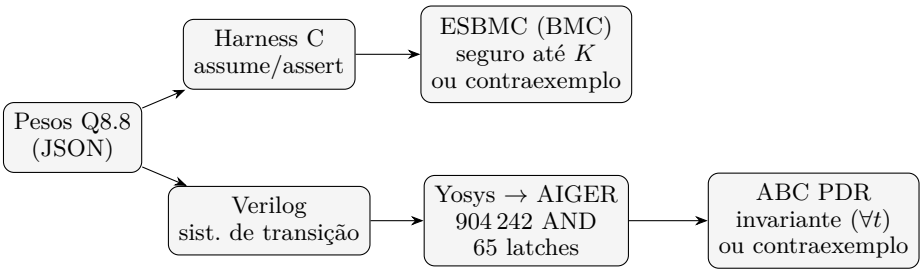
O IC3 exige um sistema de transição. A malha fechada foi exportada para Verilog (`gen_transition_system.py`), com quatro registradores de estado de 16 bits (mais um *flag* `started`, 65 *latches* no total), as mesmas expressões do ator e da dinâmica, e:

- **estado inicial**: caixa $|\text{estado}| \leq 5$ em Q8.8 ($\approx 0{,}02$), obtida por *clamp* das entradas livres no primeiro ciclo;
- **propriedade**: $\neg\,\texttt{bad}$, com $\texttt{bad} = (\theta > 53) \vee (\theta < -53)$;

- o próximo estado é **truncado** em 16 bits (`n_x[15:0]`).

Uma diferença deliberada em relação ao *harness* C: os `__ESBMC_assume` de limite por neurônio são **omitidos**, pois são uma muleta do BMC (podam o espaço); nenhum estado alcançável é descartado.

Figura 3 – As duas rotas de verificação a partir do mesmo ator quantizado.



Fonte: elaborada pelo autor, 2026.

4.4.2 Resultados sobre o ator real

Tabela 4 – BMC e IC3 sobre a malha fechada.

Método	**Modelo**	**Veredito**	**Tempo**	**Memória**
BMC (ESBMC)	sintético, K=4	Seguro até 4 passos	909 s	383 MB
IC3 (ABC)	sintético com batente (histórico)	**Provado** $\forall t$	0,37 s	trivial
IC3 (ABC)	ator DDPG real	Indeciso (*frame* 2)	1802 s	577 MB
BMC (ABC)	ator DDPG real	Indeciso (5 *frames*)	901 s	503 MB
Spacer (Z3)	ator DDPG real, inteiros	Indeciso	1800 s	—

Fonte: `ic3/EVIDENCIA.md`; `ic3/validacao/real/`. O modelo sintético com batente e seu log não estão no repositório atual e **não foram reexecutados**; é resultado histórico. A reexecução do PDR real mediu pico de 577 MB (o registro anterior citava 708 MB).

4.4.2.0.1 O sucesso sintético.

No modelo sintético com batente mecânico, o IC3 provou segurança ilimitada em 0,37 s com um invariante de quatro cláusulas sobre três dos 65 bits de estado: ele ignorou a rede e usou só a estrutura do batente. Nenhum BMC produz esse tipo de resultado. Mas, justamente porque a segurança vem da física imposta e não do controlador, esse resultado demonstra a capacidade do método, não a segurança do ator.

4.4.2.0.2 O ator real.

Nenhum motor decide. O log bruto do PDR (Apêndice A) mostra que o ABC parou no *frame* 2 com 22 cláusulas e 106 chamadas SAT. Só 42,2 s (2,34%) dos 1801 s foram contabilizados em chamadas SAT concluídas. A interpretação, uma **hipótese não medida por chamada**, é que o gargalo é uma única consulta SAT sobre a transição, que nunca termina, e não a convergência do IC3.

4.4.3 Comparação controlada sobre a mesma instância

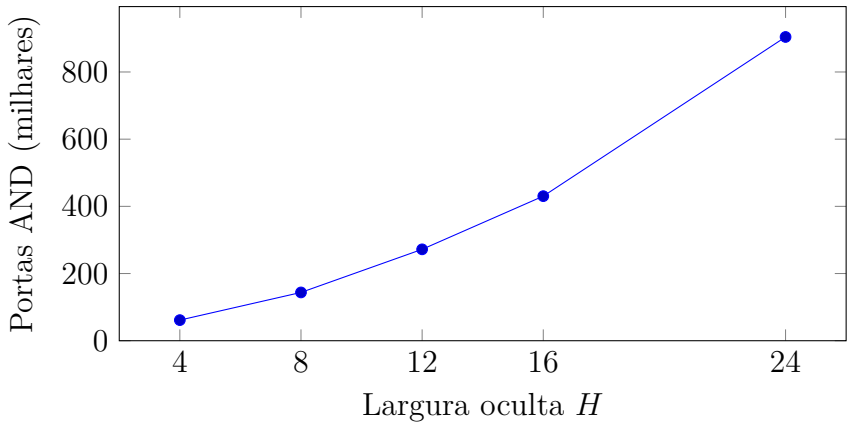
Para separar o efeito do tamanho do efeito do método, geraram-se redes aleatórias (pesos de semente fixa 7) de arquitetura $4 \to H \to H \to 1$ e rodaram-se os três motores por 300 s cada (Tabela 5, Figura 4).

Tabela 5 – Comparação controlada, mesma instância, 300 s por motor.

H	Portas AND	PDR (ABC)	BMC (ABC `bmc3`)	Spacer (Z3)
4	61 359	indeciso	9 *frames*	indeciso
8	143 721	indeciso	5 *frames*	indeciso
12	272 044	indeciso	5 *frames*	indeciso
16	430 346	indeciso	5 *frames*	indeciso
24 (real)	904 242	indeciso (1800 s)	5 *frames* (901 s)	indeciso (1800 s)

Fonte: `ic3/validacao/RESULTADOS.md`. Pesos aleatórios: não se sabe se essas instâncias são seguras.

Figura 4 – Tamanho do problema (portas AND do AIG) versus largura H das camadas ocultas.



Fonte: Tabela 5. O tamanho cresce aproximadamente com H^2.

4.4.3.0.1 O que a Tabela 5 diz.

Nenhum dos três motores decide nem a menor rede (H=4, 61 mil portas). Portanto a dificuldade *não* é só de tamanho: variar H de 4 a 24 não muda o veredito. Este é um resultado negativo sobre os motores e a codificação, e uma advertência: o sucesso sintético **não** se generaliza por escala.

4.5 Validação do pipeline

Uma prova de segurança é tão confiável quanto a tradução que a produziu. Este capítulo mede o quanto o pipeline Verilog→AIGER é confiável.

4.5.1 Fidelidade da tradução

4.5.1.1 O lema da divisão

O gerador implementa a divisão por 256 como `td(v) = v[31] ? -((-v)>>>8) : (v>>>8)`. Um problema SAT sobre todos os v de 32 bits compara `td` com o `v/256` do C. O resultado é **uma única divergência**: $v = -2^{31}$, onde `td` dá $+8\,388\,608$ e o C dá $-8\,388\,608$ (a negação de -2^{31} transborda). Para todos os outros valores as duas coincidem.

4.5.1.2 Análise de intervalos

Calculou-se, com aritmética de intervalos em inteiros, o intervalo de **759 expressões** do ator para estado de 16 bits e entrada arbitrária. O maior valor absoluto é $1{,}81 \times 10^9 < 2^{31} \approx 2{,}15 \times 10^9$: **nenhum estouro de 32 bits e nenhum argumento de `td` pode ser** -2^{31}. Logo a única divergência do lema é inalcançável. Com estado de 32 bits (variante `--bits 32`), em contraste, 749 das 759 expressões podem estourar, de modo que essa variante **não** é fiel ao `int` do C.

4.5.1.3 Equivalência por SAT

O *miter* monolítico (gerador ≡ referência com `/`) tem 9 736 668 variáveis e 28 860 715 cláusulas e **não terminou em 3000 s**. Um *miter composicional* (um por neurônio, com entradas livres restritas aos intervalos calculados) provou inicialmente **49 de 50 blocos equivalentes**: os 24 neurônios da camada 1, os 24 da camada 2 e a saída. O bloco misto `TANH_DIN` (que misturava o tanh de cinco ramos com cinco divisões da dinâmica em uma única fórmula) estourou 600 s. Ao decompor essa verificação em sub-blocos modulares (o tanh por partes, a aceleração angular e os quatro termos de integração de estado), o Yosys SAT provou **todos os 55 blocos em frações de segundo** ($< 0{,}2$ s cada). Assim, a equivalência formal da transição completa (rede + atuador + dinâmica) está **provada por SAT composicional**.

Tabela 6 – Resumo da fidelidade da tradução.

Verificação	**Resultado**
`td` vs `/256`, todo v de 32 bits	divergente só em $v = -2^{31}$
Intervalos, 759 expressões, estado de 16 bits	máx. $1{,}81 \times 10^9$; sem estouro; -2^{31} inalcançável
Intervalos, estado de 32 bits	749 podem estourar: variante infiel
Miter SAT monolítico	indeciso (3000 s)
Miter SAT composicional modular	**55/55 provados** ($< 0{,}2$ s cada)
Prova de ausência de *wrap* de 16 bits	**Provado** por SAT (0,36 s, todo o domínio)
Teste diferencial por amostragem	coincide (não é prova universal)

Fonte: `ic3/validacao/RESULTADOS.md`.

4.5.1.3.1 Ausência de *wrap* de 16 bits.

O estado é armazenado em registradores truncados de 16 bits. Uma verificação SAT independente com Yosys (`prova_wrap16.py`) provou em 0,36 s que, para qualquer estado dentro do envelope operacional admissível ($|x| \leq 24$ m, $|\dot{x}| \leq 50$ m/s, $|\theta| \leq 120°$, $|\dot{\theta}| \leq 50$ rad/s, $|F_Q| \leq 2560$), nenhuma variável de próximo estado transborda o intervalo $[-32768, 32767]$. Logo, o truncamento `[15:0]` é semanticamente idêntico ao `int` do C nos estados de interesse.

4.5.2 Teste de mutação: o pipeline sabe reprovar?

Se um verificador só devolve "indeciso", nada se sabe sobre sua capacidade de reprovar. Criaram-se mutantes do ator real com insegurança *conhecida* e rodou-se o PDR (limite de 300 s):

O ABC **reprova os três mutantes comprovadamente inseguros**. O pipeline, portanto, não só "não prova": ele acha contraexemplos quando existem em profundidade pequena ou média (para m3, no *frame* 77). Sobre m4: foi criado esperando-se insegurança, mas o controlador pode de fato mantê-lo; foi reclassificado como inválido em vez de contado.

4.5.3 Motor independente em nível de palavra

O Z3/Spacer, com aritmética inteira exata (sem bit-*blasting*), foi aplicado ao ator real (1800 s) e aos mutantes (400 s). Resultado: **indeciso em todos**, inclusive em m1, cujo contraexemplo tem profundidade 1. É um resultado negativo sobre *esta* codificação e versão (Z3 4.8.12); não se estende a Pono, AVR, btormc ou rIC3, que não foram testados. A hipótese de que "o gargalo é o bit-*blasting*" **não ganhou suporte**: mudar para nível de

Tabela 7 – Mutantes do ator real.

Mutante	**Insegurança conhecida**	**Veredito**	**Tempo**
m5: θ inicial livre	sim, profundidade 0	Contraexemplo	3,5 s
m1: $\dot{\theta}$ inicial livre	sim, profundidade 1 ($\theta_1 = \theta_0 + 640 > 53$)	Contraexemplo	6,9 s
m3: controle desligado	sim, instável em malha aberta	Contraexemplo	42,2 s
m2: atuador invertido	não sabida	Indeciso	302 s
m4: limite $\lvert\theta\rvert \leq 6$	**inválido**: o controlador pode manter $\lvert\theta\rvert \leq 6$	Indeciso	308 s

Fonte: `ic3/validacao/mut/`. m2 e m4 não têm verdade conhecida e não contam como teste.

palavra com o Z3 não ajudou, e a dificuldade parece estar na consulta de um passo (ReLU mais divisão truncada).

4.6 Resultados preliminares: discussão e ameaças à validade

4.6.1 Balanço de evidência

Tabela 8 – O que está estabelecido, o que está refutado e o que está aberto.

Afirmação	**Estado**	**Evidência**
Ator real é seguro $\forall t$ (IC3)	**Aberto**	PDR indeciso (frame 2, 1800 s)
Ator real é seguro em 2 passos, caixa larga	**Refutado**	contraexemplo em 25–29 s (ESBMC)
Pipeline reprova modelo inseguro	**Estabelecido** (3 mutantes)	Tabela 7
Rede equivalente ao C (por bloco)	**55/55 provado**	*miter* composicional modular
Tradução sem *wrap* de 16 bits	**Provado**	SAT em 0,36 s (`prova_wrap16.py`)
Planta e região inicial = planta real	**Aberto**	não validado
Prova não trivial (sem batente)	**Aberto**	não obtida
IC3 supera BMC no ator real	**Não estabelecido**	ambos indecisos

Fonte: elaborada pelo autor, 2026.

4.6.2 Isto é revolucionário?

Ainda não, e este relatório não o afirma. O resultado positivo (prova ilimitada em 0,37 s) vale para um modelo sintético com batente. No ator real a técnica não decide, nem o BMC, nem o Spacer. A literatura já tem verificação de redes quantizadas por SMT/BMC, por programação linear inteira, *model checking* neural e verificação de C via BTOR2/IC3; encontrei referências a esses temas apenas por títulos e trechos (por exemplo arXiv 2106.05997 sobre redes quantizadas, e Giacobbe et al., NeurIPS 2024), **sem leitura**. Numa busca curta não achei IC3 sobre malha fechada planta mais rede, mas ausência na busca não é ausência na literatura. Falta uma revisão sistemática e a comparação com verificadores dedicados (Marabou, NNV, α, β-CROWN), que usam propagação de limites e *branch-and-bound*.

O que *já* é contribuição defensável: (i) o método de isolar a propriedade da rede para detectar vacuidade (duas de três provas originais eram vazias); (ii) a cadeia de validação, com mutantes de verdade conhecida e *miters* composicionais; (iii) o registro honesto dos resultados negativos.

4.6.3 Ameaças à validade

- **Planta.** A dinâmica é linearizada e o estado inicial é uma caixa pequena; ambos não foram validados contra a planta real.
- **Tradução.** Estado de 16 bits truncado; ausência de *wrap* alcançável não provada; `TANH_DIN` não provado equivalente.
- **Comparação.** ESBMC (registros do repositório) e ABC/Spacer (reexecuções) não compartilham máquina; tempos absolutos não se comparam.
- **Escala.** A comparação controlada usa pesos aleatórios de segurança desconhecida.
- **Histórico.** O modelo sintético com batente e seu log não estão no repositório atual.

5 ENTREGÁVEIS

1. **Relatório/dissertação** no padrão abnTeX2 da UFAM, com os resultados, as ameaças à validade e os vereditos nas quatro categorias do Capítulo 4.
2. **Pipeline reprodutível**: geradores de *harness* C e de Verilog, *scripts* de execução (`run_pdr.sh`, validação diferencial) e `Dockerfile` com as versões fixadas das ferramentas.
3. **Conjunto de *benchmarks***: ator DDPG real, redes aleatórias de largura $H \in \{4, 8, 12, 16\}$ e mutantes com insegurança conhecida (m1, m3, m5) e reclassificados (m2, m4).
4. **Logs brutos versionados** de todas as execuções (PDR, BMC, Spacer, miters).
5. **Provas de validação**: lema da divisão, intervalos de 759 expressões, miter composicional e, a produzir, a equivalência de `TANH_DIN` por ramo e a ausência de *wrap* de 16 bits.
6. **Comparação com motor BTOR2** em nível de palavra (btormc, Pono, AVR ou rIC3) sobre o mesmo modelo.
7. **Revisão sistemática** da literatura e comparação com verificadores de redes (Marabou, NNV, α, β-CROWN).
8. **Artigo** com os resultados, a ser submetido apenas após os itens 5–7.

6 METODOLOGIA E CRONOGRAMA

6.1 Metodologia

A pesquisa é experimental e quantitativa. Cada experimento fixa a instância, o motor, a versão da ferramenta, o limite de tempo e a máquina, e registra veredito, tempo, memória e log bruto. Os critérios de rigor adotados são:

- **isolar a propriedade da rede**: executar o *assert* sem os pesos para detectar vacuidade;
- **validar o verificador**: usar mutantes de insegurança comprovada, e descartar os mutantes sem verdade conhecida;
- **comparação controlada**: mesma instância e mesmo limite para todos os motores;
- **relatar o negativo**: indeciso e *timeout* entram nos resultados com o mesmo peso de uma prova;
- **nada de generalização sem teste**: afirmações sobre motores não testados são proibidas.

6.2 Atividades

6.3 Cronograma

O cronograma abaixo é uma **proposta** em meses relativos ao início da etapa restante; as durações são estimativas do autor e devem ser ajustadas com a orientação. As atividades A1–A4 já estão concluídas.

Tabela 9 – Atividades planejadas (em ordem de custo–benefício).

N.	**Atividade**	**Situação**
A1	Formalização da malha fechada e propriedades A–D; isolamento de vacuidade	Concluída
A2	Verificação com ESBMC; envelope de 2 passos; parede em K=4	Concluída
A3	Tradução Verilog/AIGER; PDR no ator real e nos *benchmarks*	Concluída
A4	Validação: intervalos, miter composicional, mutantes, Spacer	Concluída
A5	Equivalência de `TANH_DIN` dividida por ramo	Concluída
A6	Prova de ausência de *wrap* de 16 bits (ou guarda de estouro)	Concluída
A7	Motor BTOR2 em nível de palavra sobre o mesmo modelo	A fazer
A8	Prova não trivial: invariante sem batente em modelo reduzido	A fazer
A9	Revisão sistemática e comparação com Marabou/NNV/α, β-CROWN	A fazer
A10	Validação da planta linearizada e da região inicial contra a planta não linear	A fazer
A11	Redação da dissertação e do artigo	Em andamento

Fonte: elaborada pelo autor, 2026.

Tabela 10 – Cronograma proposto das atividades restantes ("X" marca o mês de execução).

Atividade	**M1**	**M2**	**M3**	**M4**	**M5**	**M6**
A5 `TANH_DIN` por ramo	X	X				
A6 ausência de *wrap*	X	X				
A7 motor BTOR2		X	X			
A8 prova não trivial			X	X		
A9 revisão e comparação	X	X	X	X		
A10 planta não linear				X	X	
A11 redação				X	X	X

Fonte: elaborada pelo autor, 2026.

7 CONCLUSÃO

O ESBMC (BMC) decide propriedades locais e de curto horizonte do controlador DDPG quantizado e produz contraexemplos concretos e reproduzíveis, mas esbarra numa parede estrutural em $K \approx 4$ e não decide segurança para horizonte ilimitado. A rota IC3/PDR gera uma prova ilimitada num modelo com batente mecânico, mas não decide o ator real; um motor em nível de palavra (Z3/Spacer) também não. O pipeline foi validado quanto à tradução (intervalos e SAT composicional, 49/50 blocos) e quanto à capacidade de reprovar (três mutantes inseguros detectados), e as lacunas restantes estão enumeradas, com atividades e entregáveis para fechá-las.

O trabalho **não** afirma que o controlador real seja seguro para sempre, nem que o resultado seja revolucionário. Afirma o que foi provado, o que foi refutado e o que permanece indeciso, e propõe os passos para decidir o restante.

Apêndices

APÊNDICE A – LOGS BRUTOS SELECIONADOS

A.1 PDR no ator real (excerto de `real/cl_ddpg16.abc.out`)

```
ABC command line: "read_aiger cl_ddpg16.aig; pdr -v -T 1800".
Frame Clauses Max Queue Flops Cex Time
  1 : 0 21 1 14 1.50 sec
  2 : 0 14 8 2 16 1800.89 sec
Reached timeout (1800 seconds) in frame 2.
Block = 22 Oblig = 25 Clause = 22 Call = 106 (sat=61.3%) Cex = 0
SAT solving = 42.18 sec ( 2.34 %)
Property UNDECIDED. Time = 1801.36 sec
```

A.2 Resumo do *wrapper* (`real/run_pdr.stdout.txt`)

```
i/o = 65/1 lat = 65 and = 904242 lev = 689
Reached timeout (1800 seconds) in frame 2.
status=INCONCLUSIVO tempo=1802.6s pico_RSS=577MB
```

A.3 Contraexemplo do lema da divisão (`lema_td.cex.txt`)

```
SAT proof finished - model found: FAIL!
  \a 8388608 800000 00000000100000000000000000000000
  \b -8388608 ff800000 11111111100000000000000000000000
  \v -2147483648 80000000 10000000000000000000000000000000
```

A.4 Mutante detectado (`mut/mutantes.saida.txt`)

```
### m3_controle_desligado
Output 0 of miter "m3_controle_desligado" was asserted in frame 77. Time = 42.18 sec
status=CONTRAEXEMPLO tempo=42.2s pico_RSS=63MB
```

APÊNDICE B – REPRODUÇÃO

Os *scripts* e logs estão em `pibic/ic3/` e `pibic/ic3/validacao/`. A imagem de ferramentas é construída pelo `Dockerfile` da pasta de validação (Ubuntu 24.04, Yosys, Z3). Exemplos:

```
python3 gen_transition_system.py --bits 16 -o cl_ddpg16.v
python3 validate_forward.py -n 12
./run_pdr.sh cl_ddpg16.v 1800 # 0 prova, 1 cex, 2 erro, 3 indeciso
```

REFERÊNCIAS

[1] BARTO, A. G.; SUTTON, R. S.; ANDERSON, C. W. Neuronlike adaptive elements that can solve difficult learning control problems. *IEEE Trans. Systems, Man, and Cybernetics*, v. 13, n. 5, p. 834–846, 1983.

[2] BIERE, A. et al. Symbolic model checking without BDDs. In: *TACAS*, 1999.

[3] GADELHA, M. R. et al. ESBMC 5.0: an industrial-strength C model checker. In: *ASE*, 2018.

[4] BRADLEY, A. R. SAT-based model checking without unrolling. In: *VMCAI*, 2011.

[5] EÉN, N.; MISHCHENKO, A.; BRAYTON, R. Efficient implementation of property directed reachability. In: *FMCAD*, 2011.

[6] BRAYTON, R.; MISHCHENKO, A. ABC: a system for sequential synthesis and verification. In: *CAV*, 2010.

[7] WOLF, C.; GLASER, J.; KEPLER, J. Yosys: a free Verilog synthesis suite. In: *Austrochip*, 2013.

[8] KOMURAVELLI, A.; GURFINKEL, A.; CHAKI, S. SMT-based model checking for recursive programs. In: *CAV*, 2014.

[9] DE MOURA, L.; BJØRNER, N. Z3: an efficient SMT solver. In: *TACAS*, 2008.

[10] LILLICRAP, T. P. et al. Continuous control with deep reinforcement learning. In: *ICLR*, 2016.

[11] KATZ, G. et al. Reluplex: an efficient SMT solver for verifying deep neural networks. In: *CAV*, 2017.

[12] KATZ, G. et al. The Marabou framework for verification and analysis of deep neural networks. In: *CAV*, 2019.

[13] TRAN, H.-D. et al. NNV: the neural network verification tool for deep neural networks and learning-enabled cyber-physical systems. In: *CAV*, 2020.

[14] WANG, S. et al. Beta-CROWN: efficient bound propagation with per-neuron split constraints for neural network robustness verification. In: *NeurIPS*, 2021.

[15] arXiv:2106.05997 (título e autoria a conferir). Verificação de redes neurais quantizadas.

[16] GIACOBBE, M. et al. (título a conferir). In: *NeurIPS*, 2024.